

データビリティweb講座 画像処理 オプション課題 環境導入編

大倉 史生

0 はじめに

資料やコードなどのバグ報告は大倉 okura@ist.osaka-u.ac.jp まで。

0.1 オプション演習の概要

ローカルに Python + OpenCV の環境をインストールし、開発を進める。インストール等の詳細は次章で説明する。OpenCV は、Open Computer Vision Library の略であり、画像処理関連の標準的なライブラリである。Python、OpenCV とともに習得が容易なので、興味のある人はいろいろ遊んでみると良い。楽しいと思う。

0.2 参考図書

1. デジタル画像処理編集委員会, ”デジタル画像処理 [改訂第二版] ” .
画像処理関連の広い範囲の技術について、フルカラーで解説されている。
2. <https://docs.opencv.org/>.
OpenCV の公式リファレンス。チュートリアルもあるので、必要に応じて参照しよう。

1 環境の導入

1.1 前提

Python3 系、IDE として Visual Studio Code (VSCode) で説明するが、どの環境でもだいたい同じである。以下、コマンド中で `python` となっている部分は、環境によっては `python3` の場合があるので、各自の環境に合わせて使ってほしい。

Python Python をインストールしておくこと。Windows の場合、<https://python.org/downloads/> からでインストールできる (`python.exe` を PATH に追加する、的なオプションにチェックを入れておく)。最新版をインストールしておけばたぶん問題ない。ここで一度再起動する。

統合開発環境 (IDE) 統合開発環境 (IDE) は、是非この機会に導入しておこう。Python の場合、VSCode や PyCharm¹ あたりがメジャー。

¹どちらが良いかは戦争になるので言及を避ける。VSCode は様々な言語に対応しており、Python 用の拡張をインストールして使える。PyCharm は Python 専用。ざっくり言えば、高機能 & Python 特化の PyCharm と、軽量 & 高汎用性の VSCode、という感じか (これはこれで怒られそう)。ちなみに、大倉は PyCharm 派であるが、今のタイミングで初めて統合開発環境を使うなら VSCode のほうがナウいし用途が広いので良いと思う。

Git 下記で、カジュアルに「Gitでリポジトリをクローンして...」というような操作が入っている。zipでリポジトリごとダウンロードして解凍して使っても良いが、Gitまわりに触ったことがないならこの機会に是非触ってみると良い。Windowsの場合は<https://gitforwindows.org/>よりインストールを。

1.2 演習用リポジトリのクローン

本演習用のリポジトリ https://github.com/fumio125/datability_web をクローンする。以下の通り。

```
$ git clone https://github.com/fumio125/datability_web.git
```

これで、カレントディレクトリ以下 `datability_web` 内に、Git リポジトリ内のファイルがコピーされた。

1.3 必要なライブラリの導入

仮想環境の作成 Python は、pip などの管理ソフトを使って簡単にライブラリパッケージを導入できる。ただ、全体の Python 環境を汚すと後が面倒²なので、venv などのツール³を使って特定プロジェクト用の「仮想環境」を使ってから行うことが多い。以下、python3 の部分は、環境によっては python となる場合もある。

```
$ cd datability_web
$ python -m venv .venv
```

これで、Python 周りの環境（インタプリタなど）がごっそり `datability_web/.venv/` 以下にコピーされた。作成した仮想環境に含まれる Python インタプリタを使う場合は、

```
$ source .venv/bin/activate # Ubuntu 等の場合
> ./venv/Scripts/activate   # Windows の場合
```

とすると、以降は `.venv` に入っている Python 環境に対する操作になる。Windowsなどで、activate できない場合はあらかじめ

```
> Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process
```

という内容を打って、環境変数を設定しておくといい⁴。

必要なパッケージの導入 仮想環境を activate した状態で、以下のように OpenCV とその拡張パッケージ⁵、それ以外に使いそうなパッケージをインストールする。また必要になったら、必要なものをその都度入れたら良い。

```
$ python -m pip install -U pip # pip のアップデート。1 回だけやれば OK。
$ python -m pip install opencv-contrib-python # OpenCV
$ python -m pip install matplotlib, scipy, Pillow, jupyter # その他のライブラリ
```

ちなみに、今回の場合は Git リポジトリに必要なライブラリのリストが `requirements.txt` として用意されているので、以下でも OK。

```
$ python -m pip install -r requirements.txt
```

Windows などでは、pip 時に「アクセスが拒否されました」のようなエラーが出ることがある。この場合は pip のコマンドに `--user` のオプションを追加する。

²他のプロジェクトと衝突したりする。特にライブラリのバージョン指定などに齟齬があると大変。

³Python だけでなく、もっと広い範囲を仮想化するようなツール（Docker など）を使っても良い。

⁴参考資料：https://qiita.com/jugem_373/items/09c2d6abb7ccdd0fee48

⁵OpenCV の拡張パッケージが入っていないのは `opencv-python` で、原則こちらは商用利用が可能。contrib には、特許関連で商用利用しづらいアルゴリズムや、新機能なども含まれる。

1.4 IDE の設定とプログラムの実行

サンプル用のソースコードが `dataability_web/hello.py` にある。当然、任意のテキストエディタでこれらを編集し、(↑の仮想環境に `activate` した状態で) コマンドライン上で `Python` を呼べば実行できる。たとえば：

```
$ python hello.py
```

とすれば、画像が描画されたウィンドウが開く（何かキーボードから入力すれば実行終了する）。しかし、コーディングやデバッグの容易性などあらゆる面で IDE の方が優れているので、この機会に触れておこう。以下は VSCode の例だが、他の IDE でも大体同じような感じである。

基本的な流れ

1. VSCode を開き、File – Open Folder から、リポジトリのルートフォルダ (`dataability_web`) を開く。
2. 左（環境による）にある「Explorer」というビュー⁶より、`hello.py` をダブルクリック
3. View – Command Palette より、“Python: Select Interpreter” より⁷、`.venv: venv` に対応するものを選択。
4. 右上（環境による）にある再生ボタンっぽいものを押すと起動。なお、再生ボタンっぽいものの右にある矢印より、Debug を選択するとデバッガと一緒に起動できる（Run – Start Debugging からでもいい）。起動中に終了したい場合は、起動するとでてくるウィンドウから停止ボタンっぽいものを押す。

Tips

- 作業中に、VSCode のプラグインが足りないと「～をインストールしますか？」というようなことを聞かれるので、OK しておく。
- View – Terminal を開いておくと、指定されたインタプリタに対応する仮想環境に `activate` された状態の端末が開くので、`pip` とかできて便利（タブを切り替えると、ターミナルの他に、エラーやプログラムの出力なども見られる）。
- コードエディタで適当に何かを入力すれば、その先の補完案や、関数引数の定義が表示される。これがないと生きていけない。例えば、`hello.py` の適当な行で `cv2.` とでも入力してみると良い。

デバッグ IDE を使うなら、ぜひデバッガを使おう。例えば、以下のような操作が可能なのでやってみよう。他にも様々な機能があるので、ぜひ遊んでみると良い。

1. コードエディタの任意の行の左端をクリックすると、赤い丸が現れる（ブレークポイント）。試しに、`hello.py` の 12 行目にブレークポイントを置き、デバッグ起動してみよう。
2. ブレークポイントで実行が一時停止される。ブレークポイントによる停止時（あるいはエラー停止時、例外発生時など）、変数リストやコールスタック（どの関数がどの関数を呼んで今に至るかなど）を見ることができる。
3. ターミナルウィンドウ内のデバックコンソールより、適当な Python コードを入力すると実行できる。たとえば `src.shape` と入力すると、読み込まれた画像 `src` のサイズを出力できる。
4. Step Over のボタンを押すと、一行だけ実行できる。Continue を押すと、現在の場所から続きが実行される。

⁶見当たらない場合は左端のアイコンのそれらしいやつをクリック。

⁷これが見つからない場合は、Python extension をインストールする必要がある。Explorer より `.py` のファイルを適当に開くと、「インストールしますか？」のようなことを聞かれると思うので、インストールすれば OK。

2 チュートリアル

OpenCV でできるごく簡単な処理について、Jupyter notebook で書かれたチュートリアルを用意している。Jupyter notebook は、インタラクティブな環境で python を試することができるものであり、ノートブックに埋め込まれたコードスニペット（セル）ごとに実行できるため、Python プログラミングの学習や、ちょっとしたお試しプログラミング用に大変便利である⁸。各セルに記載されたコードは、通常の Python コード内でも動くので、課題プログラムの作成において、必要に応じて使用すると良い。

とりあえず、tutorial.ipynb を開いてみよう。コマンドラインからは

```
$ jupyter notebook
```

でブラウザ上で jupyter を起動し、tutorial.ipynb を開くことができる。

VSCoDe 上からは、Explorer から直接開いて編集や実行できる。以下、VSCoDe でノートブックを編集・実行する際の Tips である：

- 最初の実行時にインタプリタを選択する必要があるので、上述と同じ venv を選択すると良い。
- コードセル左上の再生ボタンっぽい何かを押すと、当該セルを実行できる。再生ボタン横の矢印より、デバッグ起動も可能。ブレークポイントも置ける。
- コードセル上で Shift + Enter すると実行し、次のセルにカーソルが合う。
- ハングして応答が帰ってこなくなったら、上部の「Restart」を押す。
- セルとセルの間にカーソルを持ってくると、Markdown（テキスト）や Code（コード）のセルを作成できる。

3 オプション課題の作成・提出

さて、いよいよ課題の作成に入ろう。課題は以下の通りである。

3.1 オプション課題

OpenCV を使って、なにか「**自分（あるいは誰か特定のユーザ）が得する**」システムを作る！

最低限の条件 下記、最低限の条件を守りながら、自由な発想で画像処理アプリを作成しよう。

- 画像処理を活用するプログラムであること。
- 諸々の資料を参考にしたり組み合わせるのは良いが、そのままコピーするのはダメ（tutorial.ipynb の内容は除く）。
- 参考にした資料があるなら、レポート中に（引用として）書いておくこと。

⁸一方、大きなプログラムを書いたりデバッグしたりするのには向かないので、ある程度大きなプログラム（例えば卒業研究や実際の研究開発に近い実装）を書く場合は、普通の Python ファイルとして作成する（あるいは、実行用のインタフェース部分だけノートブック形式にするなど）ことをオススメする。

好ましい条件

- 実験用素材（入力画像など）は、構築したアプリを良くアピールできるような例を自分で撮影すると良い。
- レポートに、うまくいく例、うまくいかない例を示し、うまく行かない例を改善するための可能性を考察する。
- しっかりした問題意識や需要に基づくアプリの提案。ただし、その問題や需要が一般的であるかどうかは一切問わない。
- ユニークな発想。

3.2 提出用コード

`option_template.py` として、課題提出用のファイルを用意した。冒頭部に、必要事項を記入しよう。

- `submission_name` に氏名を記入
- `submission_highlights` に、アピールポイントを簡潔に記入。
- 実行に準備（インストールや実行環境）が必要な場合、`submission_requirements` に記入。

あとは、適宜書き換えて課題を作成すること。

3.3 レポート

適当なドキュメント（PDF）にまとめ、`okura@ist.osaka-u.ac.jp` まで送ってください。

3.4 ヒント

何を作ればよいか思いつかない `docs` ディレクトリ以下に、ちょうど良さそうな例をいくつか紹介したスライドを用意したので、それらに取り組んでも良い。ただし、それらにとらわれず、自由な発想で取り組むことをおすすめする。

どれくらいの規模感のものを作れば良い？ 小さいもので構わない。上記のチュートリアルの内容を組み合わせた、あるいは少々書き換えただけのようなものでも、その有用性や実装内容がレポートで十分に説明されていれば問題ない。ちなみに、`tutorial.ipynb` はあくまで参考資料の一つ、という立ち位置である。よって、チュートリアルに記載されたコードを使う場合も、それ自体の説明もきちんとレポート中に含めること。

こういうのが作りたいんだけど、どのように実装すれば良いかわからない 大倉（`okura@ist.osaka-u.ac.jp`）までお気軽にご相談を。